

```
DROP TABLE IF EXISTS
bu_frd.model_behavior_features_z_2022_2025;
```

```
CREATE TABLE
bu_frd.model_behavior_features_z_2022_2025
USING PARQUET
AS
```

```
SELECT
```

```
  b.*,
```

```
/* =====
1. ACTIVE ACCOUNTS Z
Complete monthly history by design
===== */
```

```
CASE
```

```
  WHEN active_accounts_prior_sd > 0
  THEN
```

```
    (
      avg_active_accounts_recent_3m
      - avg_active_accounts_prior_9m
    )
```

```
    /
```

```
    (
      active_accounts_prior_sd / SQRT(3.0)
    )
```

```
  ELSE NULL
```

```
END AS active_accounts_z_3m,
```

```
CASE
```

```
  WHEN active_accounts_prior_sd > 0
  THEN 1 ELSE 0
```

```
END AS active_accounts_z_available_flag,
```

```
CASE
```

```
  WHEN active_accounts_prior_sd = 0
  THEN 1 ELSE 0
```

```
END AS active_accounts_prior_sd_zero_flag,
```

```
/* =====
2. BALANCE Z
Require full recent 3m + >=6 prior months
===== */
```

```
CASE
```

```
  WHEN balance_recent_months_available = 3
```

```

AND balance_prior_months_available >= 6
AND balance_prior_sd > 0
THEN
    (
        avg_balance_recent_3m
        - avg_balance_prior_9m
    )
    /
    (
        balance_prior_sd / SQRT(3.0)
    )
ELSE NULL
END AS balance_z_3m,

CASE
    WHEN balance_recent_months_available = 3
        AND balance_prior_months_available >= 6
        AND balance_prior_sd > 0
    THEN 1 ELSE 0
END AS balance_z_available_flag,

CASE
    WHEN balance_recent_months_available < 3
        OR balance_prior_months_available < 6
    THEN 1 ELSE 0
END AS balance_z_insufficient_coverage_flag,

CASE
    WHEN balance_recent_months_available = 3
        AND balance_prior_months_available >= 6
        AND balance_prior_sd = 0
    THEN 1 ELSE 0
END AS balance_prior_sd_zero_flag,

/* =====
3. AVG ACCOUNT UTILIZATION Z

utilization_z_* variables use monthly utilization
capped at TRAIN P99.9 = 1.821143
===== */

CASE
    WHEN utilization_recent_months_available = 3
        AND utilization_prior_months_available >= 6
        AND utilization_prior_sd > 0
    THEN
        (

```

```

        utilization_z_recent_avg
        - utilization_z_prior_avg
    )
    /
    (
        utilization_prior_sd / SQRT(3.0)
    )
ELSE NULL
END AS avg_utilization_z_3m,

CASE
    WHEN utilization_recent_months_available = 3
        AND utilization_prior_months_available >= 6
        AND utilization_prior_sd > 0
    THEN 1 ELSE 0
END AS avg_utilization_z_available_flag,

CASE
    WHEN utilization_recent_months_available < 3
        OR utilization_prior_months_available < 6
    THEN 1 ELSE 0
END AS avg_utilization_z_insufficient_coverage_flag,

CASE
    WHEN utilization_recent_months_available = 3
        AND utilization_prior_months_available >= 6
        AND utilization_prior_sd = 0
    THEN 1 ELSE 0
END AS avg_utilization_prior_sd_zero_flag,

```

```

/* =====
4. WEIGHTED UTILIZATION Z

weighted_util_z_* uses corrected
TRAIN P99.9 = 1.76
===== */

```

```

CASE
    WHEN weighted_util_recent_months_available = 3
        AND weighted_util_prior_months_available >= 6
        AND weighted_util_prior_sd > 0
    THEN
        (
            weighted_util_z_recent_avg
            - weighted_util_z_prior_avg
        )
    /

```

```

        (
            weighted_util_prior_sd / SQRT(3.0)
        )
    ELSE NULL
END AS weighted_utilization_z_3m,

CASE
    WHEN weighted_util_recent_months_available = 3
        AND weighted_util_prior_months_available >= 6
        AND weighted_util_prior_sd > 0
    THEN 1 ELSE 0
END AS weighted_utilization_z_available_flag,

CASE
    WHEN weighted_util_recent_months_available < 3
        OR weighted_util_prior_months_available < 6
    THEN 1 ELSE 0
END AS weighted_utilization_z_insufficient_coverage_flag,

CASE
    WHEN weighted_util_recent_months_available = 3
        AND weighted_util_prior_months_available >= 6
        AND weighted_util_prior_sd = 0
    THEN 1 ELSE 0
END AS weighted_utilization_prior_sd_zero_flag

FROM
    bu_frd.model_behavior_features_current_2022_2025 b;

```

QA 23E.1 — estructura

```

SELECT
    COUNT(*) AS rows,
    COUNT(DISTINCT party_id) AS parties,

    SUM(active_accounts_z_available_flag)
    AS active_accounts_z_available,

    SUM(balance_z_available_flag)
    AS balance_z_available,

    SUM(avg_utilization_z_available_flag)
    AS avg_util_z_available,

    SUM(weighted_utilization_z_available_flag)
    AS weighted_util_z_available

```

```
FROM bu_frd.model_behavior_features_z_2022_2025;
```

Esperamos nuevamente:

```
rows    = 688,589
```

```
parties = 688,589
```

No espero que los cuatro Z tengan 100% de disponibilidad. Eso sería incorrecto dado lo que ya vimos.

QA 23E.2 — cobertura por split y label

Este es más importante:

```
SELECT
  model_split,
  label_3m,

  COUNT(*) AS parties,

  ROUND(
    100.0 * SUM(active_accounts_z_available_flag) / COUNT(*),
    2
  ) AS active_accounts_z_available_pct,

  ROUND(
    100.0 * SUM(balance_z_available_flag) / COUNT(*),
    2
  ) AS balance_z_available_pct,

  ROUND(
    100.0 * SUM(avg_utilization_z_available_flag) / COUNT(*),
    2
  ) AS avg_util_z_available_pct,

  ROUND(
    100.0 * SUM(weighted_utilization_z_available_flag) / COUNT(*),
    2
  ) AS weighted_util_z_available_pct

FROM bu_frd.model_behavior_features_z_2022_2025

GROUP BY
  model_split,
  label_3m

ORDER BY
```

```
model_split,  
label_3m DESC;
```

Aquí quiero comprobar cuánto reduce realmente la disponibilidad nuestra regla conservadora:

```
recent = 3  
prior >= 6  
sd > 0
```

Es posible que sea bastante menor que los porcentajes de `SD > 0` que vimos anteriormente. Eso **no sería un error**; significa que estamos exigiendo comparaciones temporalmente más sólidas.

QA 23E.3 — verificar la lógica de los flags

No debería existir Z cuando `available_flag=0`, ni debería faltar cuando el flag es 1:

```
SELECT
```

```
    SUM(  
        CASE  
            WHEN active_accounts_z_available_flag = 1  
                 AND active_accounts_z_3m IS NULL  
            THEN 1 ELSE 0  
        END  
    ) AS active_flag_but_null,
```

```
    SUM(  
        CASE  
            WHEN balance_z_available_flag = 1  
                 AND balance_z_3m IS NULL  
            THEN 1 ELSE 0  
        END  
    ) AS balance_flag_but_null,
```

```
    SUM(  
        CASE  
            WHEN avg_utilization_z_available_flag = 1  
                 AND avg_utilization_z_3m IS NULL  
            THEN 1 ELSE 0  
        END  
    ) AS avg_util_flag_but_null,
```

```
    SUM(  
        CASE  
            WHEN weighted_utilization_z_available_flag = 1
```

```

        AND weighted_utilization_z_3m IS NULL
        THEN 1 ELSE 0
    END
) AS weighted_util_flag_but_null

```

```

FROM
    bu_frd.model_behavior_features_z_2022_2025;

```

Los cuatro deben ser:

0

QA 23E.4 — distribución del Z-score en TRAIN

Este QA es fundamental antes del modelo.

Aunque capped utilization reduce los outliers del numerador, todavía podemos obtener un Z-score extremo cuando el baseline SD es muy pequeño.

Por eso **no vamos a capear Z todavía**. Primero veremos qué produce realmente TRAIN.

```

SELECT

    /* ACTIVE ACCOUNTS */
    percentile_approx(
        active_accounts_z_3m,
        ARRAY(0.001,0.01,0.05,0.50,0.95,0.99,0.999)
    ) AS active_accounts_z_percentiles,

    MIN(active_accounts_z_3m)
    AS active_accounts_z_min,

    MAX(active_accounts_z_3m)
    AS active_accounts_z_max,

    /* BALANCE */
    percentile_approx(
        balance_z_3m,
        ARRAY(0.001,0.01,0.05,0.50,0.95,0.99,0.999)
    ) AS balance_z_percentiles,

    MIN(balance_z_3m)
    AS balance_z_min,

    MAX(balance_z_3m)
    AS balance_z_max,

```

```

/* AVG UTILIZATION */
percentile_approx(
    avg_utilization_z_3m,
    ARRAY(0.001,0.01,0.05,0.50,0.95,0.99,0.999)
) AS avg_util_z_percentiles,

MIN(avg_utilization_z_3m)
    AS avg_util_z_min,

MAX(avg_utilization_z_3m)
    AS avg_util_z_max,

/* WEIGHTED UTILIZATION */
percentile_approx(
    weighted_utilization_z_3m,
    ARRAY(0.001,0.01,0.05,0.50,0.95,0.99,0.999)
) AS weighted_util_z_percentiles,

MIN(weighted_utilization_z_3m)
    AS weighted_util_z_min,

MAX(weighted_utilization_z_3m)
    AS weighted_util_z_max

FROM
    bu_frd.model_behavior_features_z_2022_2025

WHERE model_split = 'TRAIN';

```